

2.4. Unified and System Memory

Heterogeneous systems have multiple physical memories where data can be stored. The host CPU has attached DRAM, and every GPU in a system has its own attached DRAM. Performance is best when data is resident in the memory of the processor accessing it. CUDA provides APIs to explicitly manage memory placement, but this can be verbose and complicate software design. CUDA provides features and capabilities aimed at easing allocation, placement, and migration of data between different physical memories.

The purpose of this chapter is to introduce and explain these features and what they mean to application developers for both functionality and performance. Unified memory has several different manifestations which depend upon the OS, driver version, and GPU used. This chapter will show how to determine which unified memory paradigm applies and how the features of unified memory behave in each. The later [chapter on unified memory](#) explains unified memory in more detail.

The following concepts will be defined and explained in this chapter:

- [Unified Virtual Address Space](#) - CPU memory and each GPU's memory have a distinct range within a single virtual address space
- [Unified Memory](#) - A CUDA feature that enables managed memory which can be automatically migrated between CPU and GPUs
 - [Limited Unified Memory](#) - A unified memory paradigm with some limitations
 - [Full Unified Memory](#) - Full support for unified memory features
 - [Full Unified Memory with Hardware Coherency](#) - Full support for unified memory using hardware capabilities
 - [Unified memory hints](#) - APIs to guide unified memory behavior for specific allocations
- [Page-locked Host Memory](#) - Non-pageable system memory, which is necessary for some CUDA operations
 - [Mapped memory](#) - A mechanism (different from unified memory) for accessing host memory directly from a kernel

Additionally, the following terms used when discussing unified and system memory are introduced here:

- [Heterogeneous Managed Memory \(HMM\)](#) - A feature of the Linux kernel that enables software coherency for full unified memory
- [Address Translation Services \(ATS\)](#) - A hardware feature, available when GPUs are connected to the CPU by the NVLink Chip-to-Chip (C2C) interconnect, which provides hardware coherency for full unified memory

2.4.1. Unified Virtual Address Space

A single virtual address space is used for all host memory and all global memory on all GPUs in the system within a single OS process. All memory allocations on the host and on all devices lie in this virtual address space. This is true whether allocations are made with CUDA APIs (e.g. `cudaMalloc`, `cudaMallocHost`) or with system allocation APIs (e.g. `new`, `malloc`, `mmap`). The CPU and each GPU has a unique range within the unified virtual address space.

This means:

- The location of any memory (that is, CPU or which GPU's memory it lies in) can be determined from the value of a pointer using `cudaPointerGetAttributes()`
- The `cudaMemcpyKind` parameter of `cudaMemcpy*()` can be set to `cudaMemcpyDefault` to automatically determine the copy type from the pointers

2.4.2. Unified Memory

Unified memory is a CUDA memory feature which allows memory allocations called *managed memory* to be accessed from code running on either the CPU or the GPU. Unified memory was shown in [the intro to CUDA in C++](#). Unified memory is available on all systems supported by CUDA.

On some systems, managed memory must be explicitly allocated. *Managed memory can be explicitly allocated* in CUDA in a few different ways:

- The CUDA API `cudaMallocManaged`
- The CUDA API `cudaMallocFromPoolAsync` with a pool created with `allocType` set to `cudaMemAllocationTypeManaged`
- Global variables with the `__managed__` specifier (see [Memory Space Specifiers](#))

On systems with [HMM](#) or [ATS](#), all system memory is implicitly managed memory, regardless of how it is allocated. No special allocation is needed.

2.4.2.1. Unified Memory Paradigms

The features and behavior of unified memory vary between operating systems, kernel versions on Linux, GPU hardware, and the GPU-CPU interconnect. The form of unified memory available can be determined by using `cudaDeviceGetAttribute` to query a few attributes:

- `cudaDevAttrConcurrentManagedAccess` - 1 for full unified memory support, 0 for limited support
- `cudaDevAttrPageableMemoryAccess` - 1 means all system memory is fully-supported unified memory, 0 means only memory explicitly allocated as managed memory is fully-supported unified memory
- `cudaDevAttrPageableMemoryAccessUsesHostPageTables` - Indicates the mechanism of CPU/GPU coherence: 1 is hardware, 0 is software.

[Figure 18](#) illustrates how to determine the unified memory paradigm visually and is followed by a [code sample](#) implementing the same logic.

There are four paradigms of unified memory operation:

- [Full support for explicit managed memory allocations](#)
- [Full support for all allocations with software coherence](#)
- [Full support for all allocations with hardware coherence](#)
- [Limited unified memory support](#)

When full support is available, it can either require explicit allocations, or all system memory may implicitly be unified memory. When all memory is implicitly unified, the coherence mechanism can either be software or hardware. Windows and some Tegra devices have limited support for unified memory.

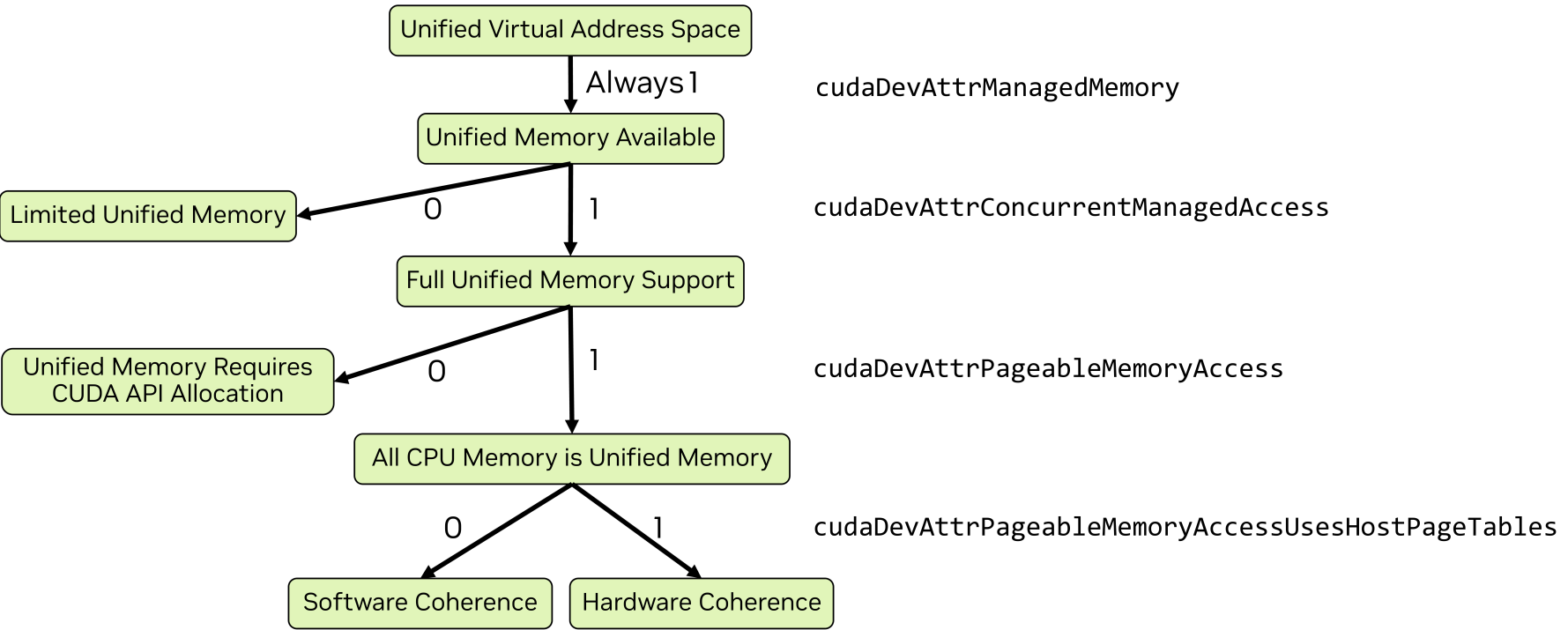


Figure 18: All current GPUs use a unified virtual address space and have unified memory available. When `cudaDevAttrConcurrentManagedAccess` is 1, full unified memory support is available, otherwise only limited support is available. When full support is available, if `cudaDevAttrPageableMemoryAccess` is also 1, then all system memory is unified memory. Otherwise, only memory allocated with CUDA APIs (such as `cudaMallocManaged`) is unified memory. When all system memory is unified, `cudaDevAttrPageableMemoryAccessUsesHostPageTables` indicates whether coherence is provided by hardware (when value is 1) or software (when value is 0).

Table 3 shows the same information as Figure 18 as a table with links to the relevant sections of this chapter and more complete documentation in a later section of this guide.

Table 3: Overview of Unified Memory Paradigms

Unified Memory Paradigm	Device Attributes	Full Documentation
Limited unified memory support	<code>cudaDevAttrConcurrentManagedAccess</code> is 0	Unified Memory on Windows, WSL, and Tegra CUDA for Tegra Memory Management Unified memory on Tegra
Full support for explicit managed memory allocations	<code>cudaDevAttrPageableMemoryAccess</code> is 0 and <code>cudaDevAttrConcurrentManagedAccess</code> is 1	Unified Memory on Devices with only CUDA Managed Memory Support
Full support for all allocations with software coherence	<code>cudaDevAttrPageableMemoryAccessUsesHostPageTables</code> is 0 and <code>cudaDevAttrPageableMemoryAccess</code> is 1 and <code>cudaDevAttrConcurrentManagedAccess</code> is 1	Unified Memory on Devices with Full CUDA Unified Memory Support
Full support for all allocations with hardware coherence	<code>cudaDevAttrPageableMemoryAccessUsesHostPageTables</code> is 1 and <code>cudaDevAttrPageableMemoryAccess</code> is 1 and <code>cudaDevAttrConcurrentManagedAccess</code> is 1	Unified Memory on Devices with Full CUDA Unified Memory Support

2.4.2.1.1. Unified Memory Paradigm: Code Example

The following code example demonstrates querying the device attributes and determining the unified memory paradigm, following the logic of Figure 18, for each GPU in a system.

```

void queryDevices()
{
    int numDevices = 0;
    cudaGetDeviceCount(&numDevices);
    for(int i=0; i<numDevices; i++)
    {
        cudaSetDevice(i);
        cudaInitDevice(0, 0, 0);
        int deviceId = i;

        int concurrentManagedAccess = -1;
        cudaDeviceGetAttribute (&concurrentManagedAccess, cudaDevAttrConcurrentManagedAccess, deviceId);
        int pageableMemoryAccess = -1;
        cudaDeviceGetAttribute (&pageableMemoryAccess, cudaDevAttrPageableMemoryAccess, deviceId);
        int pageableMemoryAccessUsesHostPageTables = -1;
        cudaDeviceGetAttribute (&pageableMemoryAccessUsesHostPageTables, cudaDevAttrPageableMemoryAccessUsesHostPageTables, deviceId);

        printf("Device %d has ", deviceId);
        if(concurrentManagedAccess){
            if(pageableMemoryAccess){
                printf("full unified memory support");
                if( pageableMemoryAccessUsesHostPageTables)
                    { printf(" with hardware coherency\n"); }
                else
                    { printf(" with software coherency\n"); }
            }
            else
                { printf("full unified memory support for CUDA-made managed allocations\n"); }
        }
        else
            { printf("limited unified memory support: Windows, WSL, or Tegra\n"); }
    }
}

```

2.4.2.2. Full Unified Memory Feature Support

Most Linux systems have full unified memory support. If device attribute `cudaDevAttrPageableMemoryAccess` is 1, then all system memory, whether allocated by CUDA APIs or system APIs, operates as unified memory with full feature support. This includes file-backed memory allocations created with `mmap`.

If `cudaDevAttrPageableMemoryAccess` is 0, then only memory allocated as managed memory by CUDA behaves as unified memory. Memory allocated with system APIs is not managed and is not necessarily accessible from GPU kernels.

In general, for unified allocations with full support:

- Managed memory is usually allocated in the memory space of the processor where it is first touched
- Managed memory is usually migrated when it is used by a processor other than the processor where it currently resides
- Managed memory is migrated or accessed at the granularity of memory pages (software coherence) or cache lines (hardware coherence)
- Oversubscription is allowed: an application may allocate more managed memory than is physically available on the GPU

Allocation and migration behavior can deviate from the above. This can be influenced by the programmer using hints and prefetches. Full coverage of full unified memory support can be found in Unified Memory on Devices with Full CUDA Unified Memory Support.

2.4.2.2.1. Full Unified Memory with Hardware Coherency

On hardware such as Grace Hopper and Grace Blackwell, where an NVIDIA CPU is used and the interconnect between the CPU and GPU is NVLink Chip-to-Chip (C2C), address translation services (ATS) are available.

`cudaDevAttrPageableMemoryAccessUsesHostPageTables` is 1 when ATS is available.

With ATS, in addition to full unified memory support for all host allocations:

- GPU allocations (e.g. `cudaMalloc`) can be accessed from the CPU (`cudaDevAttrDirectManagedMemAccessFromHost` will be 1)
- The link between CPU and GPU supports native atomics (`cudaDevAttrHostNativeAtomicSupported` will be 1)
- Hardware support for coherence can improve performance compared to software coherence

ATS provides all capabilities of HMM. When ATS is available, HMM is automatically disabled. Further discussion of hardware vs. software coherency is found in [CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency](#).

2.4.2.2.2. HMM - Full Unified Memory with Software Coherency

Heterogeneous Memory Management (HMM) is a feature available on Linux operating systems (with appropriate kernel versions) which enables software-coherent full unified memory support. Heterogeneous memory management brings some of the capabilities and convenience provided by ATS to PCIe-connected GPUs.

On Linux with at least Linux Kernel 6.1.24, 6.2.11, or 6.3 or later, heterogeneous memory management (HMM) may be available. The following command can be used to find if the addressing mode is HMM.

```
$ nvidia-smi -q | grep Addressing
Addressing Mode : HMM
```

When HMM is available, full unified memory is supported and all system allocations are implicitly unified memory. If a system also has ATS, HMM is disabled and ATS is used, since ATS provides all the capabilities of HMM and more.

2.4.2.3. Limited Unified Memory Support

On Windows, including Windows Subsystem for Linux (WSL), and on some Tegra systems, a limited subset of unified memory functionality is available. On these systems, managed memory is available, but migration between CPU and GPUs behaves differently.

- Managed memory is first allocated in the CPU's physical memory
- Managed memory is migrated in larger granularity than virtual memory pages
- Managed memory is migrated to the GPU when the GPU begins executing
- The CPU must not access managed memory while the GPU is active
- Managed memory is migrated back to the CPU when the GPU is synchronized
- Oversubscription of GPU memory is not allowed
- Only memory explicitly allocated by CUDA as managed memory is unified

Full coverage of this paradigm can be found in [Unified Memory on Windows, WSL, and Tegra](#).

2.4.2.4. Memory Advise and Prefetch

The programmer can provide hints to the NVIDIA Driver managing unified memory to help it maximize application performance. The CUDA API `cudaMemAdvise` allows the programmer to specify properties of allocations that affect where they are placed and whether or not the memory is migrated when accessed from another device.

`cudaMemPrefetchAsync` allows the programmer to suggest an asynchronous migration of a specific allocation to a different location be started. A common use is starting the transfer of data a kernel will use before the kernel is launched. This enables the copy of data to occur while other GPU kernels are executing.

The section on [Performance Hints](#) covers the different hints that can be passed to `cudaMemAdvise` and shows examples of using `cudaMemPrefetchAsync`.

2.4.3. Page-Locked Host Memory

In introductory code examples, `cudaMallocHost` was used to allocate memory on the CPU. This allocates **page-locked memory** (also known as **pinned memory**) on the host. Host allocations made through traditional allocation mechanisms like `malloc`, `new`, or `mmap` are **not page-locked**, which **means they may be swapped to disk or physically relocated by the operating system**.

Page-locked host memory is required for **asynchronous copies between the CPU and GPU**. Page-locked host memory also **improves performance** of synchronous copies. Page-locked memory can be **mapped** to the GPU for direct access from GPU kernels.

The CUDA runtime provides APIs to allocate page-locked host memory or to page-lock existing allocations:

- `cudaMallocHost` **allocates page-locked host memory**
- `cudaHostAlloc` defaults to the same behavior as `cudaMallocHost`, but also **takes flags** to specify other memory parameters
- `cudaFreeHost` **freets memory** allocated with `cudaMallocHost` or `cudaHostAlloc`
- `cudaHostRegister` **page-locks a range of existing memory** allocated outside the CUDA API, such as with `malloc` or `mmap`

`cudaHostRegister` enables host memory allocated by 3rd party libraries or other code outside of a developer's control to be page-locked so that it can be used in asynchronous copies or mapped.

Note

Page-locked host memory can be used for asynchronous copies and mapped-memory by all GPUs in the system.

Page-locked host memory is not cached on non I/O coherent Tegra devices. Also, `cudaHostRegister()` is not supported on non I/O coherent Tegra devices.

2.4.3.1. Mapped Memory

On systems with HMM or ATS, all host memory is directly accessible from the GPU using the host pointers. When ATS or HMM are not available, host allocations can be made accessible to the GPU by mapping the memory into the GPU's memory space. Mapped memory is always page-locked.

The code examples which follow will illustrate the following array copy kernel operating directly on mapped host memory.

```
__global__ void copyKernel(float* a, float* b)
{
    int idx = threadIdx.x + blockDim.x * blockIdx.x;
    a[idx] = b[idx];
}
```

While mapped memory **may be useful** in some cases where **certain data which is not copied to the GPU** needs to be accessed from a kernel, **accessing mapped memory in a kernel requires transactions across the CPU-GPU interconnect, PCIe, or NVLink C2C**. These operations **have higher latency and lower bandwidth** compared to accessing device memory. **Mapped memory should not be considered a performant alternative to unified memory or explicit memory management** for the majority of a kernel's memory needs.

2.4.3.1.1. cudaMallocHost and cudaHostAlloc

Host memory allocated with `cudaHostMalloc` or `cudaHostAlloc` is automatically mapped. The pointers returned by these APIs can be directly used in kernel code to access the memory on the host. The host memory is accessed over the CPU-GPU interconnect.

cudaMallocHost

cudaAllocHost

```
void usingMallocHost() {
    float* a = nullptr;
    float* b = nullptr;

    CUDA_CHECK(cudaMallocHost(&a, vLen*sizeof(float)));
    CUDA_CHECK(cudaMallocHost(&b, vLen*sizeof(float)));

    initVector(b, vLen);
    memset(a, 0, vLen*sizeof(float));

    int threads = 256;
    int blocks = vLen/threads;
    copyKernel<<<blocks, threads>>>(a, b);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    printf("Using cudaMallocHost: ");
    checkAnswer(a,b);
}
```

2.4.3.1.2. cudaHostRegister

When ATS and HMM are not available, allocations made by system allocators can still be mapped for access directly from GPU kernels using `cudaHostRegister`. Unlike memory created with CUDA APIs, however, the memory cannot be accessed from the kernel using the host pointer. A pointer in the device's memory region must be obtained using `cudaHostGetDevicePointer()`, and that pointer must be used for accesses in kernel code.

```
void usingRegister() {
    float* a = nullptr;
    float* b = nullptr;
    float* devA = nullptr;
    float* devB = nullptr;

    a = (float*)malloc(vLen*sizeof(float));
    b = (float*)malloc(vLen*sizeof(float));
    CUDA_CHECK(cudaHostRegister(a, vLen*sizeof(float), 0));
    CUDA_CHECK(cudaHostRegister(b, vLen*sizeof(float), 0));

    CUDA_CHECK(cudaHostGetDevicePointer((void**)&devA, (void*)a, 0));
    CUDA_CHECK(cudaHostGetDevicePointer((void**)&devB, (void*)b, 0));

    initVector(b, vLen);
    memset(a, 0, vLen*sizeof(float));

    int threads = 256;
    int blocks = vLen/threads;
    copyKernel<<<blocks, threads>>>(devA, devB);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    printf("Using cudaHostRegister: ");
    checkAnswer(a, b);
}
```

2.4.3.1.3. Comparing Unified Memory and Mapped Memory

Mapped memory makes CPU memory accessible from the GPU, but does not guarantee that all types of access, for example atomics, are supported on all systems. Unified memory guarantees that all access types are supported.

Mapped memory remains in CPU memory, which means all GPU accesses must go through the connection between the CPU and GPU: PCIe or NVLink. Latency of accesses made across these links are significantly higher than access to GPU memory, and total available bandwidth is lower. As such, using mapped memory for all kernel memory accesses is unlikely to fully utilize GPU computing resources.

Unified memory is most often migrated to the physical memory of the processor accessing it. After the first migration, repeated access to the same memory page or cache line by a kernel can utilize the full GPU memory bandwidth.

Note

Mapped memory has also been referred to as *zero-copy* memory in previous documents.

Prior to all CUDA applications using a [unified virtual address space](#), additional APIs were needed to enable memory mapping (`cudaSetDeviceFlags` with `cudaDeviceMapHost`). These APIs are no longer needed.

Atomic functions (see [Atomic Functions](#)) operating on mapped host memory are not atomic from the point of view of the host or other GPUs.

CUDA runtime requires that 1-byte, 2-byte, 4-byte, 8-byte, and 16-byte naturally aligned loads and stores to host memory initiated from the device are preserved as single accesses from the point of view of the host and other devices. On some platforms, atomics to memory may be broken by the hardware into separate load and store operations. These component load and store operations have the same requirements on preservation of naturally aligned accesses. The CUDA runtime does not support a PCI Express bus topology where a PCI Express bridge splits 8-byte naturally aligned operations and NVIDIA is not aware of any topology that splits 16-byte naturally aligned operations.

2.4.4. Summary



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2007-2026, NVIDIA Corporation & affiliates. All rights reserved.

Last updated on Mar 04, 2026.